

คู่มือทำ Lab 09–10

Authentication & JWT

ขั้นตอนละเอียดสำหรับโปรเจกต์ KUSHOP ตั้งแต่สร้างตาราง members ในฐานข้อมูล เขียน backend และ frontend ไปจนถึงทดสอบระบบสมัครสมาชิกเข้าสู่ระบบ และออกจากระบบด้วย JWT ใน cookie

โค้ดทั้งหมดในเอกสารนี้ดึงมาจากไฟล์จริงในโปรเจกต์ ณ วันที่สร้างเอกสาร
Node.js ^22.18.0 || >=24.12.0 · Express · PostgreSQL · Vue 3 · Pinia

สารบัญ

- ส่วน A** สรุปสิ่งที่ทำไปแล้วใน Lab 09–10
- ส่วน B** สิ่งที่ต้องทำเอง (ทีละขั้น)
- ส่วน C** ติดตั้งบนเครื่องใหม่หลัง clone / pull จาก GitHub

รายละเอียดแต่ละเรื่อง

1. ภาพรวมระบบ
2. สิ่งที่ต้องเตรียม
3. ฐานข้อมูล: สร้างตาราง members และให้สิทธิ์
4. ตั้งค่าไฟล์ .env
5. Backend: memberController, memberRoute, server.js
6. ทดสอบ Backend ด้วย Postman
7. Frontend: Pinia, authStore, Router
8. Frontend: หน้า Login, Register, PageMember และ MainMenu
9. รันและทดสอบทั้งระบบ
10. ปัญหาที่พบบ่อยและวิธีแก้
11. สิ่งที่ต่างจากเอกสารแลป

A สิ่งที่ทำไปแล้วใน Lab 09-10

ส่วนนี้สรุปว่าในโปรเจกต์มีอะไรทำเสร็จแล้วบ้าง **ไม่ต้องเขียนโค้ดเหล่านี้ซ้ำ** โค้ดเต็มอยู่ในหัวข้อ 5, 7 และ 8

A.1 Lab 09 – Authentication

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
<code>postMember</code> : ตรวจสอบข้อมูลครบ, ตรวจสอบอีเมลซ้ำ, hash รหัสผ่านด้วย <code>bcrypt (salt 11)</code> , INSERT ลง <code>members</code>	<code>controllers/memberController.js</code>
<code>loginMember</code> : หาอีเมล แล้วเทียบรหัสด้วย <code>bcrypt.compare</code>	<code>controllers/memberController.js</code>
route <code>POST /members</code> และ <code>POST /members/login</code>	<code>src/router/memberRoute.js</code>
เรียกใช้ route ในเซิร์ฟเวอร์	<code>server.js</code>
ฟอร์ม Login (ใช้อีเมลเป็น <code>loginName</code>) + แสดงผลสำเร็จ/ผิดพลาด	<code>src/components/TheLogin.vue</code>
ฟอร์ม Register (อีเมล, ชื่อ, รหัสผ่าน) ที่ <code>/register</code>	<code>src/components/TheRegister.vue</code>
ปุ่ม "ลงทะเบียนใหม่" ในหน้า Login และ route <code>/register</code>	<code>TheLogin.vue</code> , <code>src/router/index.js</code>
backend ส่ง <code>regist / login</code> กับ <code>message</code> กลับไป	<code>controllers/memberController.js</code>

A.2 Lab 10 – Authorization JWT

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
ติดตั้ง <code>jsonwebtoken</code> , <code>cookie-parser</code> , <code>pinia</code>	<code>package.json</code>
CORS กำหนด origin + <code>credentials: true</code>	<code>server.js</code>
<code>SECRET_KEY</code> ใน <code>.env</code> (สุ่มค่าไว้ให้แล้ว)	<code>.env</code>
login สำเร็จ: <code>jwt.sign</code> อายุ 1 ชม. แล้วตั้ง cookie <code>token</code> (<code>httpOnly</code> , <code>secure</code> , <code>sameSite: strict</code>)	<code>controllers/memberController.js</code>
login ผิด: <code>res.clearCookie('token')</code>	<code>controllers/memberController.js</code>
<code>app.use(cookieParser())</code> ก่อน router	<code>server.js</code>
<code>getMember</code> → GET <code>/members/detail</code> (<code>jwt.verify</code>)	<code>memberController.js</code> , <code>memberRoute.js</code>
<code>logoutMember</code> → GET <code>/members/logout</code>	<code>memberController.js</code> , <code>memberRoute.js</code>
<code>axios.defaults.withCredentials = true</code>	<code>src/main.js</code>
หน้า <code>PageMember</code> แสดงอีเมล, ชื่อ, <code>dutyId</code>	<code>src/components/PageMember.vue</code>
login สำเร็จแล้วไป <code>/pagemember</code>	<code>src/components/TheLogin.vue</code>
เมนูแสดงชื่อสมาชิก + ปุ่มออกจากระบบ (ถามยืนยันก่อน)	<code>src/components/MainMenu.vue</code>
Pinia + <code>authStore</code> (<code>isLogin</code> , <code>login</code> , <code>logout</code>)	<code>src/main.js</code> , <code>src/stores/authStore.js</code>
ถ้า login แล้วเข้า <code>/login</code> ให้เปลี่ยนหน้า	<code>src/router/index.js</code>

A.3 ส่วนที่ทำเพิ่มหรือต่างจากแลป

- ใช้ `bcryptjs` ที่มีอยู่แล้วแทน `bcrypt` (วิธีเรียกใช้เหมือนกัน)
- ตอนสมัครสำเร็จไม่ส่งรหัสผ่านกลับไปใน response และตรวจว่าต้องมี `password` ด้วย
- server ไม่ยอมเปิดถ้าไม่มี `SECRET_KEY`
- เช็กสถานะล็อกอินที่ router ที่เดียว แทนการให้ทุกหน้าเรียก `/members/detail` เอง
- ถ้า DB ล่มแล้ว error ไม่มีข้อความ จะส่ง `Server error` แทน
- ลบหน้าโปรไฟล์และแท็บเปลี่ยนรหัสผ่านเดิม (ผูกกับตาราง `users`) — `/profile` จะพาไป `/pagemember`
- ทดสอบแล้วผ่านด้วย token ที่เซ็นเอง (เครื่องที่พัฒนาไม่มี DB) ได้ผลตามหัวข้อ 6

ตารางเทียบแบบละเอียดพร้อมเหตุผลอยู่ในหัวข้อ 11

B สิ่งที่ต้องทำเอง (ทีละขั้น)

ยังขาดแค่ฝั่งฐานข้อมูล

โค้ดทั้งหมดพร้อมแล้ว แต่ต้องสร้างตาราง `members` และตั้งค่าการเชื่อมต่อก่อน ถึงจะสมัครสมาชิกและเข้าสู่ระบบได้จริง

ขั้นที่ 1 — เตรียม PostgreSQL (หัวข้อ 2, 3.1)

- ติดตั้งและเปิด PostgreSQL + pgAdmin 4
- ตรวจว่ามีฐานข้อมูล `kushop` ที่มีตาราง `products/brands` จากแลปก่อนหน้า

ขั้นที่ 2 — สร้าง user dev (หัวข้อ 3.2, ข้ามได้ถ้าใช้ postgres)

SQL (รันในฐานะ postgres)

sql

```
CREATE ROLE dev WITH LOGIN PASSWORD '1234';
GRANT CONNECT ON DATABASE kushop TO dev;
GRANT USAGE ON SCHEMA public TO dev;
```

ขั้นที่ 3 — สร้างตาราง members (หัวข้อ 3.3–3.4)

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.members (
  "memEmail" VARCHAR(100) NOT NULL PRIMARY KEY,
  "memName" VARCHAR(100),
  "dutyId" VARCHAR(100) DEFAULT 'member',
  "memHash" VARCHAR(255)
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members TO dev;
```

ต้องมีเครื่องหมาย " ครอบชื่อคอลัมน์ ไม่อย่างนั้นจะเจอ `column "memEmail" does not exist`

ขั้นที่ 4 — ตรวจสอบตาราง (หัวข้อ 3.5)

SQL

sql

```
SELECT column_name, data_type, column_default
FROM information_schema.columns WHERE table_name = 'members';
```

ต้องได้ 4 แถว

ขั้นที่ 5 — เติมค่าเชื่อมต่อใน .env (หัวข้อ 4)

`.env` มี `PORT` และ `SECRET_KEY` อยู่แล้ว ห้ามลบ ให้เพิ่มเฉพาะค่าที่ไม่ตรงกับค่าเริ่มต้น (`localhost:5432`, `kushop`, `postgres/postgres`)

.env (เพิ่มต่อท้าย)

env

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234
```

ขั้นที่ 6 — รันระบบ (หัวข้อ 9.1)

2 terminal ในไฟล์เดสก์ท็อป

bash

```
npm run server    # terminal 1 → http://localhost:3000
npm run dev        # terminal 2 → http://localhost:5173
```

- ถ้า server ตัวเก่าเปิดอยู่ ให้ปิด (Ctrl+C) แล้วเปิดใหม่ เพื่อให้อ่าน .env ใหม่
- เปิดเว็บด้วย `http://localhost:5173` เท่านั้น ห้ามใช้ `127.0.0.1` เพราะ cookie จะไม่ถูกส่ง

ขั้นที่ 7 — ทดสอบ API ด้วย Postman (หัวข้อ 6, ไม่บังคับ)

Request	Body	ผลที่ควรได้
POST /members	<pre>{"memEmail": "test1@gmail.com", "memName": "Mr.First", "password": "1234"}</pre>	Regist Success
POST /members/login	<pre>{"loginName": "test1@gmail.com", "password": "1234"}</pre>	Login Success + cookie token
GET /members/detail	—	ข้อมูลสมาชิก, login: true
GET /members/logout	—	เรียก detail เข้าได้ No member

ขั้นที่ 8 — ทดสอบบนหน้าเว็บ (หัวข้อ 9.2)

- 1 เปิดเว็บ → ถูกพาไป /login
- 2 กด "ลงทะเบียนใหม่" แล้วสมัคร → ใน pgAdmin ต้องมีแถวใหม่ และ memHash ขึ้นต้นด้วย \$2
- 3 ล็อกอิน → ไป /pagemember และเมนูแสดงชื่อ
- 4 F12 → Application → Cookies → มี token ที่ติ๊ก HttpOnly
- 5 รีเฟรชแล้วยังล็อกอินอยู่ และพิมพ์ /login แล้วถูกพากลับ /pagemember
- 6 กดออกจากระบบ → กลับ /login และ cookie หายไป

ติดปัญหา?

ดูตารางอาการ สาเหตุ และวิธีแก้ในหัวข้อ 10

C ติดตั้งบนเครื่องใหม่ (หลัง clone / pull)

ใช้ตอนย้ายโปรเจกต์ไปเครื่องอื่น เช่น คอมที่แลปหรือโน้ตบุ๊ก GitHub เก็บเฉพาะโค้ด **สิ่งที่ไม่ได้ติดไปด้วย** มีดังนี้:

สิ่งที่ไม่อยู่ใน git	เหตุผล	ต้องทำอะไร
node_modules/	ไฟล์ใหญ่ ติดตั้งใหม่ได้จาก package.json	npm install (ขั้นที่ C.3)
.env	มีรหัสผ่าน DB และ SECRET_KEY ห้ามขึ้น GitHub	สร้างใหม่ (ขั้นที่ C.4)
ฐานข้อมูล kushop	อยู่ใน PostgreSQL ของแต่ละเครื่อง ไม่ใช่ไฟล์ในโปรเจกต์	ย้ายหรือสร้างใหม่ (ขั้นที่ C.5)

ส่วนรูปสินค้า (img_pd/) และคู่มือนี้ (docs/) อยู่ใน git แล้ว ได้มาพร้อมโค้ด

C.1 ติดตั้งโปรแกรมบนเครื่องใหม่

โปรแกรม	ตรวจด้วยคำสั่ง
Git	git --version
Node.js ^22.18.0 >=24.12.0	node -v
PostgreSQL + pgAdmin 4	เปิด pgAdmin แล้วเชื่อมต่อ server ได้

C.2 เอาโค้ดลงเครื่อง

ครั้งแรก (เครื่องนี้ยังไม่มีโปรเจกต์) — เปิด terminal ในโฟลเดอร์ที่ต้องการ:

Terminal bash

```
git clone https://github.com/Deepciony/WebT.git
cd WebT
```

มีโปรเจกต์อยู่แล้ว — ดึงเวอร์ชันล่าสุด:

Terminal (ในโฟลเดอร์ WebT) bash

```
git checkout main
git pull origin main
```

ถ้า git pull ไม่ยอมเพราะมีไฟล์ที่แก้ค้างไว้

ดูว่าแก้อะไรไว้ด้วย `git status` ถ้าต้องการเก็บไว้ให้ `git stash` ก่อน แล้ว `git pull` จากนั้น `git stash pop`

C.3 ติดตั้ง package

```
Terminal bash
npm install
```

จะได้ทุกอย่างที่ Lab 09–10 ใช้ (`bcryptjs` , `jsonwebtoken` , `cookie-parser` , `pinia`) ไม่ต้องติดตั้งทีละตัว ต้องรันใหม่ทุกครั้ง pull แล้ว `package.json` เปลี่ยน

C.4 สร้างไฟล์ .env

- 1 สร้างไฟล์ชื่อ `.env` ที่โฟลเดอร์ `WebT` (ระดับเดียวกับ `server.js`)
- 2 สุ่ม `SECRET_KEY` ด้วยคำสั่งด้านล่าง แล้วคัดลอกผลลัพธ์
- 3 ใส่ค่าตามตัวอย่าง แล้วแก้ `DB_USER` / `DB_PASSWORD` ให้ตรงกับ PostgreSQL ของเครื่องนี้

```
Terminal bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

```
.env env
PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=postgres
DB_PASSWORD=รหัสผ่าน-postgres-ของเครื่องนี้

SECRET_KEY=วางค่าที่สุ่มได้ตรงนี้
```

SECRET_KEY แต่ละเครื่องต่างกันได้

คนที่ล็อกอินไว้กับเครื่องหนึ่ง แค่ต้องล็อกอินใหม่บนอีกเครื่อง ข้อมูลสมาชิกในฐานข้อมูลไม่ได้รับผลกระทบ ห้าม commit ไฟล์ `.env` (มีอยู่ใน `.gitignore` แล้ว)

C.5 เตรียมฐานข้อมูลบนเครื่องใหม่

โปรเจกต์ต้องใช้ตาราง `products` , `brands` , `pdTypes` (จากแลปก่อนหน้า) และ `members` (Lab 09) เลือกทำแบบใดแบบหนึ่ง:

แบบ 1 — ย้ายฐานข้อมูลจากเครื่องเดิม (แนะนำ ได้ข้อมูลสินค้าและสมาชิกครบ)

```
เครื่องเดิม: export ออกเป็นไฟล์ bash
pg_dump -U postgres -d kushop -f kushop.sql
```

คัดลอกไฟล์ `kushop.sql` ไปเครื่องใหม่ (แฟลชไดรฟ์ / ไดรฟ์ออนไลน์) อย่า commit ขึ้น GitHub เพราะมี hash รหัสผ่านสมาชิกอยู่

เครื่องใหม่: สร้างฐานข้อมูลแล้ว import

bash

```
createdb -U postgres kushop
psql -U postgres -d kushop -f kushop.sql
```

ถ้าใช้ pgAdmin แทน: เครื่องเดิมคลิกขวา `kushop` → **Backup...** เครื่องใหม่สร้างฐานข้อมูล `kushop` แล้วคลิกขวา → **Restore...**

แบบ 2 — สร้างใหม่

1. สร้างฐานข้อมูล `kushop` และตารางสินค้าตามเอกสารแลปก่อนหน้า
2. สร้างตาราง `members` และให้สิทธิ์ตามส่วน B ขั้นที่ 2–4 (สมาชิกเดิมจะไม่มี ต้องสมัครใหม่)

คำสั่ง `pg_dump` / `psql` หาไม่เจอ

เพิ่มโฟลเดอร์ `bin` ของ PostgreSQL ใน PATH (เช่น `C:\Program Files\PostgreSQL\17\bin`) หรือพิมพ์ path เต็ม หรือใช้เมนู Backup/Restore ใน pgAdmin แทน

C.6 รันโปรเจกต์

2 terminal ในโฟลเดอร์ WebT

bash

```
npm run server    # terminal 1 → API server is running at http://localhost:3000
npm run dev       # terminal 2 → Local: http://localhost:5173/
```

เปิด `http://localhost:5173` แล้วทดสอบตามส่วน B ขั้นที่ 8

C.7 ครั้งต่อไปที่ pull อัปเดต

Terminal

bash

```
git pull origin main
npm install          # ถ้า package.json เปลี่ยน
# ปิด npm run server (Ctrl+C) แล้วเปิดใหม่ เพื่อให้โค้ด backend ใหม่มีผล
```

ฝั่ง `npm run dev` อัปเดตหน้าเว็บให้อัตโนมัติ ไม่ต้องปิดเปิดใหม่ ถ้า commit ใหม่มีการเปลี่ยนโครงสร้างตาราง ต้องแก้ฐานข้อมูลตามด้วย

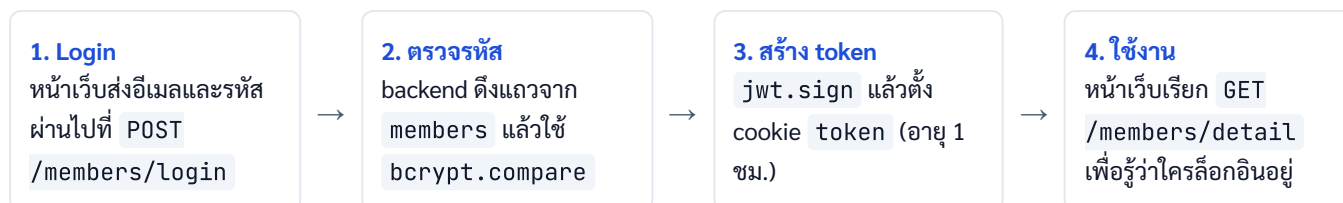
C.8 ปัญหาที่เจอบ่อยบนเครื่องใหม่

อาการ	สาเหตุ	วิธีแก้
PowerShell: npm.ps1 cannot be loaded because running scripts is disabled	Windows ปิดการรันสคริปต์	ใช้ Command Prompt แทน หรือรัน <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
Cannot find module 'jsonwebtoken' (หรือ package อื่น)	ยังไม่ได้ <code>npm install</code> หลัง pull	รัน <code>npm install</code>
server ไม่ขึ้น: SECRET_KEY is missing	ไม่มีไฟล์ <code>.env</code> หรือไฟล์ผิดที่	ทำขั้นที่ C.4 ไฟล์ต้องอยู่ข้าง <code>server.js</code> ชื่อ <code>.env</code> ไม่มี <code>.txt</code> ต่อท้าย
EADDRINUSE: address already in use :::3000	มีโปรแกรมอื่นใช้ port 3000	ปิดโปรแกรมนั้น (หน้าเว็บเรียก API ที่ port 3000 ตายตัว จึงไม่ควรเปลี่ยน PORT)
Vite เปิดที่ 5174 แทน 5173 แล้ว ล็อกอินไม่ได้	port 5173 ถูกใช้อยู่ และ CORS อนุญาตแค่ 5173	ปิดโปรแกรมที่ใช้ 5173 หรือรัน <code>npm run dev -- --port 5173 --strictPort</code>
หน้าสินค้าว่าง / "ยังโหลดสินค้าไม่ได้"	ฐานข้อมูลบนเครื่องใหม่ยังไม่มี ตารางสินค้า	ทำขั้นที่ C.5
ล็อกอินด้วยบัญชีเดิมไม่ได้	สร้างฐานข้อมูลใหม่ (แบบ 2) จึงไม่มี สมาชิกเดิม	สมัครใหม่ หรือย้ายฐานข้อมูลแบบ 1
<code>git push</code> ได้ 403 / Permission denied	Windows จำบัญชี GitHub อื่นไว้	Credential Manager → ลบ <code>git:https://github.com</code> แล้ว push ใหม่ด้วยบัญชีเจ้าของ repo

1 ภาพรวมระบบ

Lab 09 (Authentication) ทำระบบสมัครสมาชิกและเข้าสู่ระบบ โดยเก็บรหัสผ่านเป็นค่า hash ของ bcrypt ในตาราง members

Lab 10 (Authorization JWT) เมื่อเข้าสู่ระบบสำเร็จ backend จะสร้าง JWT แล้วส่งกลับใน cookie แบบ httpOnly จากนั้นทุกคำขอของ browser จะแนบ cookie ไปด้วย backend จึงรู้ว่าใครใช้งานอยู่ โดยไม่ต้องเก็บสถานะไว้ที่ server (stateless)



Endpoint	หน้าที่	ผลลัพธ์
POST /members	สมัครสมาชิก	{ message, regist }
POST /members/login	เข้าสู่ระบบ + ตั้ง cookie	{ message, login }
GET /members/detail	ถอด token จาก cookie	{ memEmail, memName, dutyId, login }
GET /members/logout	ลบ cookie	{ message, login: false }

ไฟล์ที่เกี่ยวข้อง

ฝั่ง	ไฟล์	หน้าที่
DB	ตาราง members	เก็บข้อมูลสมาชิก
Backend	.env	ค่าเชื่อมต่อ DB และ SECRET_KEY
Backend	controllers/memberController.js	สมัคร, login, detail, logout
Backend	src/router/memberRoute.js	ผูก URL กับ controller
Backend	server.js	CORS, cookie-parser, เรียกใช้ route
Frontend	src/main.js	ติดตั้ง Pinia, ส่ง cookie ทุกคำขอ
Frontend	src/stores/authStore.js	เก็บสถานะการล็อกอิน
Frontend	src/router/index.js	กำหนดหน้าที่ต้องล็อกอิน
Frontend	TheLogin.vue , TheRegister.vue , PageMember.vue , MainMenu.vue	หน้าจอ

2 สิ่งที่ต้องเตรียม

โปรแกรม	ใช้ทำอะไร
Node.js ^22.18.0 >=24.12.0	รัน backend และ Vite
PostgreSQL + pgAdmin 4	ฐานข้อมูล kushop ที่มีตาราง products/brands จากแลปก่อนหน้า
Postman (ไม่บังคับ)	ทดสอบ API ก่อนต่อกับหน้าเว็บ
Google Chrome / Edge	ทดสอบหน้าเว็บ และดู cookie ใน DevTools

ติดตั้ง package

package ของ Lab 09–10 อยู่ใน `package.json` แล้ว ถ้าเพิ่ง clone โปรเจกต์มา ให้รันคำสั่งนี้ครั้งเดียวในโฟลเดอร์โปรเจกต์:

```
Terminal bash
npm install
```

ถ้าทำเองจากโปรเจกต์เปล่า ให้ติดตั้งทีละตัวตามแลป:

```
Terminal bash
npm install bcryptjs          # hash รหัสผ่าน (Lab 09)
npm install jsonwebtoken      # สร้าง/ตรวจ JWT (Lab 10)
npm install cookie-parser     # อ่าน cookie ผัง backend (Lab 10)
npm install pinia             # state management ผัง Vue (Lab 10)
```

ทำไมใช้ bcryptjs ไม่ใช่ bcrypt

โปรเจกต์นี้มี `bcryptjs` อยู่แล้ว วิธีเรียกใช้ (`hash` , `compare`) เหมือน `bcrypt` ในเอกสารแลปทุกอย่าง และไม่
ต้องคอมไพล์ native module บน Windows

3 ฐานข้อมูล: ตาราง members

ทำใน pgAdmin 4 ตามเอกสาร Lab 09 หน้า 2 หรือรันคำสั่ง SQL ด้านล่างแทนก็ได้ ผลเหมือนกัน

3.1 ตรวจสอบว่ามีฐานข้อมูล kushop

- 1 เปิด pgAdmin แล้วเชื่อมต่อ server PostgreSQL
- 2 คูณใน **Databases** ว่ามี `kushop` อยู่ ถ้ายังไม่มี คลิกขวา **Databases** → **Create** → **Database...** ตั้งชื่อ `kushop`
- 3 คลิกขวาที่ `kushop` → **Query Tool** เพื่อเปิดหน้าต่างรัน SQL

3.2 (ถ้าจะใช้ user dev) สร้าง login role

เอกสารแลบใช้ user `dev` รหัส `1234` ถ้ามีอยู่แล้วข้ามขั้นนี้ได้ ถ้าจะใช้ `postgres` ก็ข้ามได้เช่นกัน

SQL (รันในฐาน `postgres`)

sql

```
CREATE ROLE dev WITH LOGIN PASSWORD '1234';
GRANT CONNECT ON DATABASE kushop TO dev;
GRANT USAGE ON SCHEMA public TO dev;
```

3.3 สร้างตาราง members ด้วย pgAdmin

- 1 ในฐานข้อมูล `kushop` ขยาย **Schemas** → **public** คลิกขวา **Tables** → **Create** → **Table...**
- 2 แท็บ **General**: Name = `members`, Owner = `postgres`, Schema = `public`
- 3 แท็บ **Columns**: กด + เพิ่ม 4 คอลัมน์ตามตารางด้านล่าง
- 4 แท็บ **Security**: กด + ที่ Privileges เลือก Grantee = `dev` แล้วติ๊ก INSERT, SELECT, UPDATE, DELETE
- 5 กด **Save**

Name	Data type	Length	Not NULL?	Primary key?	Default
<code>memEmail</code>	character varying	100	✓	✓	
<code>memName</code>	character varying	100			
<code>dutyId</code>	character varying	100			<code>'member'</code>
<code>memHash</code>	character varying	255			

3.4 หรือสร้างด้วย SQL

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.members (
  "memEmail" VARCHAR(100) NOT NULL PRIMARY KEY, -- ใช้เป็นชื่อล็อกอิน
  "memName" VARCHAR(100),
  "dutyId" VARCHAR(100) DEFAULT 'member', -- สิทธิ์การใช้งาน
```

```
"memHash" VARCHAR(255) -- hash ของรหัสผ่าน (bcrypt)
);

-- ให้ user ที่แอปใช้เชื่อมต่อ อ่าน/เขียนตารางได้ (ข้ามได้ถ้าใช้ postgres)
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members TO dev;
```

ต้องมีเครื่องหมาย " รอบชื่อคอลัมน์

PostgreSQL จะแปลงชื่อที่ไม่มี " เป็นตัวพิมพ์เล็กทั้งหมด (memEmail → mememail) แต่ controller query ด้วย "memEmail" ถ้าสร้างผิด จะเจอ error column "memEmail" does not exist

3.5 ตรวจสอบผล

SQL

sql

```
SELECT column_name, data_type, column_default
FROM information_schema.columns
WHERE table_name = 'members'
ORDER BY ordinal_position;

SELECT * FROM members; -- ดอนนี้ยังว่าง
```

ผลที่ควรได้

คำสั่งแรกต้องได้ 4 แถว: memEmail, memName, dutyId (default 'member'::character varying), memHash

4 ตั้งค่าไฟล์ .env

สร้างไฟล์ `.env` ที่โฟลเดอร์หลักของโปรเจกต์ (ระดับเดียวกับ `server.js`) ไฟล์นี้อยู่ใน `.gitignore` แล้ว จะไม่ถูก commit

```
.env env
PORT=3000

# การเชื่อมต่อฐานข้อมูล (ชื่อตัวแปรตาม database.js ของโปรเจกต์นี้)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234

# กฎแล็บสำหรับเซ็น JWT
SECRET_KEY=ใส่ค่าสุ่มยาวๆ-ที่นี่
```

ชื่อตัวแปรฐานข้อมูลในโปรเจกต์นี้ต่างจากเอกสารแลป (`DBSERVER` , `DBUSER` ...) เพราะ `database.js` อ่านค่าชื่อ `DB_*` ถ้าไม่ใช่ จะใช้ค่าเริ่มต้นดังนี้:

```
database.js js
import pg from "pg";
import dotenv from "dotenv";

dotenv.config();

const { Pool } = pg;

const database = new Pool({
  host: process.env.DB_HOST || "localhost",
  port: Number(process.env.DB_PORT || 5432),
  database: process.env.DB_NAME || "kushop",
  user: process.env.DB_USER || "postgres",
  password: process.env.DB_PASSWORD || "postgres"
});

export default database;
```

สร้าง SECRET_KEY แบบสุ่ม

```
Terminal bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

ห้ามเปิดเผย SECRET_KEY

ใครรู้ค่านี สร้าง token ปลอมเป็นสมาชิกคนไหนก็ได้ อย่าใช้ค่าต่างๆ อย่าง thailandgogo ในงานจริง และอย่า commit ไฟล์ .env ถ้า server เปิดโดยไม่มี SECRET_KEY จะหยุดทำงานพร้อมข้อความ SECRET_KEY is missing

5 Backend

5.1 controllers/memberController.js

ฟังก์ชัน	แลป	ทำอะไร
postMember	09	ตรวจสอบข้อมูลครบ → ตรวจสอบอีเมลซ้ำ → hash รหัสผ่าน (salt 11) → INSERT
loginMember	09 + 10	หาอีเมล → bcrypt.compare → สำเร็จ: jwt.sign + res.cookie / ล้มเหลว: res.clearCookie
getMember	10	อ่าน req.cookies.token → jwt.verify → ส่งข้อมูลสมาชิก
logoutMember	10	res.clearCookie('token')

controllers/memberController.js

js

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import database from "../database.js";

// httpOnly keeps the token away from page JavaScript (XSS); strict stops cross-site sends.
const cookieOptions = {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
};

export async function postMember(req, res) {
  console.log(`POST /members is requested.`);
  const bodyData = req.body;
  try {
    if (!bodyData.memEmail || !bodyData.memName || !bodyData.password) {
      return res.json({ message: `ERROR memEmail, memName and password are required.`,
regist: false });
    }
    const chkRow = await database.query({
      text: `SELECT * FROM members WHERE "memEmail" = $1`,
      values: [bodyData.memEmail]
    });
    if (chkRow.rowCount !== 0) {
      return res.json({ message: `ERROR memEmail ${bodyData.memEmail} is exists.`, regist:
false });
    }

    // Store only the bcrypt hash, never the password itself
    const saltround = 11;
    const pwdHash = await bcrypt.hash(bodyData.password, saltround);
    await database.query({
      text: `INSERT INTO "members" ("memEmail", "memName", "memHash")
VALUES ($1, $2, $3)`,
      values: [bodyData.memEmail, bodyData.memName, pwdHash]
    });
  }
```

```

    // Echo back only public fields, not the password
    res.json({
      memEmail: bodyData.memEmail,
      memName: bodyData.memName,
      createDate: new Date(),
      message: "Regist Success",
      regist: true
    });
  }
  catch (err) {
    return res.json({ message: err.message || `Server error`, regist: false });
  }
}

export async function loginMember(req, res) {
  console.log(`POST /members/login is requested.`);
  const bodyData = req.body;
  try {
    if (!bodyData.loginName || !bodyData.password) {
      return res.json({ message: `Login and Password is required`, login: false });
    }
    const result = await database.query({
      text: `SELECT * FROM members WHERE "memEmail" = $1`,
      values: [bodyData.loginName]
    });
    if (result.rowCount == 0) {
      return res.json({ message: `Login Fail`, login: false });
    }

    const loginOK = await bcrypt.compare(bodyData.password, result.rows[0].memHash);
    if (loginOK) {
      // Token payload: identifying but not secret
      const theuser = {
        memEmail: result.rows[0].memEmail,
        memName: result.rows[0].memName,
        dutyId: result.rows[0].dutyId
      };
      const token = jwt.sign(theuser, process.env.SECRET_KEY, { expiresIn: '1h' });
      res.cookie('token', token, { ...cookieOptions, maxAge: 3600000 });
      res.json({ message: `Login Success`, login: true });
    }
    else {
      res.clearCookie('token', cookieOptions);
      res.json({ message: `Login Fail`, login: false });
    }
  }
  catch (err) {
    return res.json({ message: err.message || `Server error`, login: false });
  }
}

export async function getMember(req, res) {
  console.log(`GET /members/detail is requested.`);
  const token = req.cookies.token;
  if (!token)
    return res.json({ message: `No member`, login: false });
  try {
    // verify fails if the token or its signature was tampered with

```

```

    const member = jwt.verify(token, process.env.SECRET_KEY);
    return res.json({
      memEmail: member.memEmail,
      memName: member.memName,
      dutyId: member.dutyId,
      login: true
    });
  }
  catch (err) {
    console.log(err.message);
    return res.json({ message: `The information was falsified`, login: false });
  }
}

export async function logoutMember(req, res) {
  console.log(`GET /members/logout is requested.`);
  try {
    res.clearCookie('token', cookieOptions);
    res.json({ message: `Logout Success`, login: false });
  }
  catch (err) {
    return res.json({ message: err.message });
  }
}
}

```

ทำไมไม่ส่ง HTTP status 4xx

ตามเอกสารแลป: axios มอง status 4xx เป็น error แล้วกระโดดไป `catch` เพื่อให้เขียน frontend ง่าย backend จึงตอบ 200 เสมอ แล้วบอกผลผ่าน `login / regist` แทนงานจริงควรส่ง status ให้ถูกต้อง

5.2 src/router/memberRoute.js

```

src/router/memberRoute.js  js

import express from "express";
import * as memberC from "../../controllers/memberController.js";

const router = express.Router();

router.get("/members/detail", memberC.getMember);
router.get("/members/logout", memberC.logoutMember);
router.post("/members", memberC.postMember);
router.post("/members/login", memberC.loginMember);

export default router;

```

5.3 server.js

จุดที่เพิ่มจากแลป:

- **CORS** แบบระบุโดเมน พร้อม `credentials: true` เพื่อให้ browser ส่ง cookie ข้าม port ได้
- `app.use(cookieParser())` ต้องเรียก ก่อน router ไม่อย่างนั้น `req.cookies` จะว่าง
- `app.use(memberRoutes)`

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import cookieParser from "cookie-parser";
import productRoutes from "./src/router/productRoute.js";
import authRoutes from "./src/router/authRoute.js";
import databaseRoutes from "./src/router/databaseRoute.js";
import memberRoutes from "./src/router/memberRoute.js";

dotenv.config();

if (!process.env.SECRET_KEY) {
  throw new Error("SECRET_KEY is missing. Add it to .env before starting the API.");
}

const app = express();
const PORT = Number(process.env.PORT || 3000);

// Whitelist the frontend and allow cookies to travel with requests
app.use(cors({
  origin: ["http://localhost:5173", "http://127.0.0.1:5173"],
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));
app.use(express.json());
app.use("/img_pd", express.static("img_pd"));
// Must run before the routers so req.cookies is filled in
app.use(cookieParser());
app.use(productRoutes);
app.use(authRoutes);
app.use(databaseRoutes);
app.use(memberRoutes);

app.listen(PORT, () => {
  console.log(`API server is running at http://localhost:${PORT}`);
});
```

6 ทดสอบ Backend ด้วย Postman

เปิด backend ก่อนด้วย `npm run server` (ต้องเห็น `API server is running at http://localhost:3000`) จากนั้นใน Postman ตั้ง Body เป็น **raw** → **JSON**

6.1 สมัครสมาชิก

POST <code>http://localhost:3000/members</code>	json
<pre>{ "memEmail": "test1@gmail.com", "memName": "Mr.First", "password": "1234" }</pre>	
ผลที่ควรได้	json
<pre>{ "memEmail": "test1@gmail.com", "memName": "Mr.First", "createDate": "2026-...", "message": "Regist Success", "regist": true }</pre>	

ส่งซ้ำอีกครั้งต้องได้ `"ERROR memEmail test1@gmail.com is exists."` และใน pgAdmin คอลัมน์ `memHash` ต้องขึ้นต้นด้วย `$2` (เป็น hash ไม่ใช้รหัสผ่านจริง)

6.2 เข้าสู่ระบบ

POST <code>http://localhost:3000/members/login</code>	json
<pre>{ "loginName": "test1@gmail.com", "password": "1234" }</pre>	
ผลที่ควรได้	json
<pre>{ "message": "Login Success", "login": true }</pre>	

ดูที่แท็บ **Cookies** ของ response ต้องมี cookie ชื่อ `token` (ค่าขึ้นต้น `eyJ`) ถ้ารหัสผิดจะได้ `{ "message": "Login Fail", "login": false }`

6.3 อ่านข้อมูลจาก token

GET <code>http://localhost:3000/members/detail</code>	http
(ไม่ต้องมี body – Postman ส่ง cookie token ที่ได้จากข้อ 6.2 ให้อัตโนมัติ)	

ผลที่ควรได้

json

```
{
  "memEmail": "test1@gmail.com",
  "memName": "Mr.First",
  "dutyId": "member",
  "login": true
}
```

กรณี	ผลลัพธ์
ไม่มี cookie	{ "message": "No member", "login": false }
แก้ค่า token แม้แต่ตัวเดียว	{ "message": "The information was falsified.", "login": false }
token หมดอายุ (เกิน 1 ชม.)	{ "message": "The information was falsified.", "login": false }

6.4 ออกจากระบบ

GET http://localhost:3000/members/logout

json

```
{ "message": "Logout Success", "login": false }
```

หลังจากนี้เรียก /members/detail อีกครั้งต้องได้ No member

7 Frontend: Pinia, authStore, Router

7.1 src/main.js

ติดตั้ง Pinia และตั้ง `axios.defaults.withCredentials = true` **ครั้งเดียวที่นี่** (ในแลปตั้งซ้ำทุก component) ถ้าไม่ตั้ง browser จะไม่ส่ง/รับ cookie ข้าม port

src/main.js

js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import axios from 'axios'
import App from './App.vue'
import router from './router'
import './assets/main.css'
import './assets/skylearn.css'
import './stores/theme.js'

// Send the auth cookie with every API request
axios.defaults.withCredentials = true

const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

7.2 src/stores/authStore.js

ตามแลปมี `isLogin`, `login()`, `logout()` และเพิ่ม:

- `member` เก็บข้อมูลจาก `/members/detail` ให้ทุก component ใช้ร่วมกัน (แทนที่แต่ละหน้าจะเรียก API เอง)
- `getMember()` เรียก `/members/detail` แล้วอัปเดตสถานะ
- `memLogout()` เรียก `/members/logout` แล้วล้างสถานะ
- `checked` บอกว่าเช็ค cookie ไปแล้วหรือยังในการเปิดหน้าครั้งนี้
- `fakeLogin()` ใช้กับปุ่ม "ขำมการเข้าสู่ระบบ" ตอนพัฒนาเท่านั้น

src/stores/authStore.js

js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'

export const useAuthStore = defineStore('auth', () => {
  const isLogin = ref(false)
  const member = ref(null)
  const checked = ref(false)

  const login = () => {
    isLogin.value = true
  }

  const logout = () => {
```

```

    isLogin.value = false
    member.value = null
  }

  // Ask the API to decode the httpOnly token cookie
  const getMember = async () => {
    try {
      const res = await axios.get('http://localhost:3000/members/detail')
      isLogin.value = Boolean(res.data.login)
      member.value = res.data.login ? res.data : null
    } catch (err) {
      console.log(err.message)
      logout()
    }
    checked.value = true
  }

  // Clears the httpOnly cookie on the server, then the local state
  const memLogout = async () => {
    try {
      await axios.get('http://localhost:3000/members/logout')
    } catch (err) {
      console.log(err.message)
    }
    logout()
  }

  // Dev-only: pretend to be signed in when no API/DB is running
  const fakeLogin = () => {
    member.value = { memEmail: 'dev@localhost', memName: 'Dev', dutyId: 'member', login: true }

    isLogin.value = true
    checked.value = true
  }

  return { isLogin, member, checked, login, logout, getMember, memLogout, fakeLogin }
})

```

7.3 src/router/index.js

- เพิ่ม route `/register` (TheRegister) และ `/pagemember` (PageMember)
- `meta.requiresAuth` = ต้องล็อกอินก่อน, `meta.guestOnly` = เฉพาะคนที่ยังไม่ล็อกอิน
- `beforeEach` เช็ก cookie ผ่าน `authStore.getMember()` ครั้งที่เปิดเว็บ แล้วตัดสินใจว่าจะให้ไปไหนไหน นี่คือนั่นคือ ส่วน "ถ้า Login แล้ว เข้าหน้า Login ให้เปลี่ยนหน้า" ของ Lab 10 หน้า 15

src/router/index.js

js

```

import { createRouter, createWebHistory } from 'vue-router';
import TheHome from '../components/TheHome.vue';
import TheProduct from '../components/PageProduct.vue';
import TheLogin from '../components/TheLogin.vue';
import TheRegister from '../components/TheRegister.vue';
import PageMember from '../components/PageMember.vue';
import ProductManage from '../components/ProductManage.vue';
import DatabaseOverview from '../components/DatabaseOverview.vue';
import CartPage from '../components/CartPage.vue';

```

```
import { useAuthStore } from '../stores/authStore.js';
```

```
const routes = [  
  {  
    path: '/',  
    name: 'Home',  
    component: TheHome,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/product',  
    name: 'Product',  
    component: TheProduct,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/manage',  
    name: 'ProductManage',  
    component: ProductManage,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/pagemember',  
    name: 'PageMember',  
    component: PageMember,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/profile',  
    redirect: '/pagemember',  
  },  
  {  
    path: '/database',  
    name: 'DatabaseOverview',  
    component: DatabaseOverview,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/cart',  
    name: 'Cart',  
    component: CartPage,  
    meta: { requiresAuth: true },  
  },  
  {  
    path: '/login',  
    name: 'Login',  
    component: TheLogin,  
    meta: { guestOnly: true },  
  },  
  {  
    path: '/register',  
    name: 'Register',  
    component: TheRegister,  
    meta: { guestOnly: true },  
  },  
];
```

```
const router = createRouter({
```

```
    history: createWebHistory(),
    routes,
  });

  router.beforeEach(async (to) => {
    const authStore = useAuthStore();
    // Check the token cookie once per page load; login/logout keep the store in sync after that
    if (!authStore.checked) await authStore.getMember();

    if (to.meta.requiresAuth && !authStore.isLogin) {
      return { name: 'Login' };
    }

    // Already signed in: skip the login/register pages
    if (to.meta.guestOnly && authStore.isLogin) {
      return { name: 'PageMember' };
    }
  });

  export default router;
```

8 Frontend: หน้าจอ

8.1 TheLogin.vue

ส่ง loginName (อีเมล) และ password ไป /members/login ถ้าสำเร็จเรียก authStore.login() + getMember() แล้วไป /pagemember ตาม Lab 10 หน้า 13

src/components/TheLogin.vue

vue

```
<template>
  <AuthLayout :heading="t('auth.welcome')" :highlight="t('auth.back')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('auth.loginTitle') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="loginName" type="email" required
autofocus autocomplete="username"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="current-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('auth.submitLogin') }} <span></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !login }">{{
t(message) }}</p>
    <router-link to="/register" class="auth-switch-link">{{ t('auth.toRegister') }}</router-
link>
    <DevBypass v-if="DevBypass" />
  </AuthLayout>
</template>

<script setup>
import { defineAsyncComponent, ref } from 'vue'
import { useRouter } from 'vue-router'
import axios from 'axios'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import AuthLayout from './AuthLayout.vue'

const DevBypass = import.meta.env.DEV ? defineAsyncComponent(() => import('../DevBypass.vue')) :
null

const authStore = useAuthStore()
const router = useRouter()
const loginName = ref('')
const password = ref('')
const login = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {
    const response = await axios.post('http://localhost:3000/members/login', {
      loginName: loginName.value,
      password: password.value
    })
  }
```

```

    login.value = response.data.login
    message.value = response.data.message
    if (login.value) {
      authStore.login()
      await authStore.getMember()
      router.push('/pagemember')
    }
  } catch (err) {
    console.log(err)
    login.value = false
    message.value = 'auth.loginFail'
  } finally {
    busy.value = false
  }
}
</script>

```

AuthLayout.vue

กรอบหน้า (โลโก้, ปุ่มสลับภาษา/ธีม, พื้นหลัง) แยกไว้ใน `AuthLayout.vue` เพื่อใช้ร่วมกับหน้า Register ส่วน `t(...)` คือฟังก์ชันแปลภาษาจาก `src/i18n.js`

8.2 TheRegister.vue

ส่ง `memEmail`, `memName`, `password` ไป `POST /members` แล้วแสดงผลจาก `regist` และ `message` ตาม Lab 09 หน้า 10–11

`src/components/TheRegister.vue`

vue

```

<template>
  <AuthLayout :heading="t('register.heading')" :highlight="t('register.highlight')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('register.title') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="memEmail" type="email" required
autofocus autocomplete="email"></label>
      <label>{{ t('register.name') }}<input v-model.trim="memName" type="text" required
autocomplete="name"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="new-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('register.submit') }} <span></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !regist }">
{{ t(message) }}</p>
    <router-link to="/login" class="auth-switch-link">{{ t('register.toLogin') }}</router-
link>
  </AuthLayout>
</template>

<script setup>
import { ref } from 'vue'
import axios from 'axios'
import { t } from '../i18n.js'
import AuthLayout from './AuthLayout.vue'

```

```

const memEmail = ref('')
const memName = ref('')
const password = ref('')
const regist = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {
    const response = await axios.post('http://localhost:3000/members', {
      memEmail: memEmail.value,
      memName: memName.value,
      password: password.value
    })
    regist.value = response.data.regist
    message.value = response.data.message
    if (regist.value) password.value = ''
  } catch (err) {
    console.log(err)
    regist.value = false
    message.value = 'register.fail'
  } finally {
    busy.value = false
  }
}
</script>

```

8.3 PageMember.vue

แสดงข้อมูลจาก token ที่ backend ถอดให้ (อีเมล ชื่อ dutyId) ตาม Lab 10 หน้า 9 โดยอ่านจาก `authStore.member`

src/components/PageMember.vue (ไม่รวม CSS)

vue

```

<template>
  <section class="member-page">
    <div class="member-card sk-panel">
      <div class="member-avatar" aria-hidden="true">{{ initials }}</div>
      <p class="member-kicker sk-kicker">{{ t('profile.kicker') }}</p>
      <h1>{{ member?.memName }}</h1>

      <dl class="member-details">
        <div><dt>{{ t('auth.email') }}</dt><dd>{{ member?.memEmail }}</dd></div>
        <div><dt>{{ t('member.duty') }}</dt><dd><span class="sk-pill">{{ member?.dutyId
|| '-' }}</span></dd></div>
        <div><dt>{{ t('profile.status') }}</dt><dd class="member-online">{{
t('member.signedIn') }}</dd></div>
      </dl>

      <router-link to="/product" class="member-shop sk-btn sk-btn-block">{{
t('home.browse') }} </router-link>
      <button class="member-logout sk-btn sk-btn-coral sk-btn-block" type="button"
@click="logout">{{ t('profile.logout') }}</button>
    </div>
  </section>
</template>

```

```

<script setup>
import { computed } from 'vue'
import { useRouter } from 'vue-router'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'

const authStore = useAuthStore()
const router = useRouter()
const member = computed(() => authStore.member)
const initials = computed(() => (member.value?.memName || 'KU').slice(0, 2).toUpperCase())

const logout = async () => {
  if (!window.confirm(t('member.confirmLogout'))) return
  await authStore.memLogout()
  router.push('/login')
}
</script>

```

8.4 MainMenu.vue

แสดงชื่อสมาชิก (ลิงก์ไป PageMember) และปุ่มออกจากระบบเมื่อ `authStore.isLogin` เป็น true ถ้ายังไม่ล็อกอินแสดงลิงก์ Login ตาม Lab 10 หน้า 10–11 และ 14

src/components/MainMenu.vue – ส่วน script

vue

```

<script setup>
import { computed } from 'vue'
import { useRouter } from 'vue-router'
import { cartCount } from '../stores/cart.js'
import { isSky } from '../stores/theme.js'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import SkyIcon from './SkyIcon.vue'
import AppSwitches from './AppSwitches.vue'

const authStore = useAuthStore()
const router = useRouter()
const memName = computed(() => authStore.member?.memName)

const memLogout = async () => {
  if (!window.confirm(t('member.confirmLogout'))) return
  await authStore.memLogout()
  router.push('/login')
}

const links = [
  { to: '/', label: 'nav.home', icon: 'home' },
  { to: '/product', label: 'nav.products', icon: 'shop' },
  { to: '/login', label: 'nav.login', icon: 'login', guest: true },
  { to: '/manage', label: 'nav.manage', icon: 'pencil', member: true },
  { to: '/database', label: 'nav.database', icon: 'database', member: true }
]

const visibleLinks = computed(() => links.filter((link) =>
  authStore.isLogin ? !link.guest : !link.member
))

```

```
))  
</script>
```

src/components/MainMenu.vue – รายการเมนู (เริ่มคลาสสิก)

vue

```
<li v-if="authStore.isLogin" class="nav-item">  
  <router-link to="/pagemember" style="text-decoration: none"><div class="nav-link fw-bold">{{  
    memName }}</div></router-link>  
</li>  
<li v-if="authStore.isLogin" class="nav-item">  
  <a href="#" style="text-decoration: none" @click.prevent="memLogout"><div class="nav-link fw-  
    bold">{{ t('profile.logout') }}</div></a>  
</li>  
<li v-if="!authStore.isLogin" class="nav-item">  
  <router-link to="/login" style="text-decoration: none"><div class="nav-link">{{ t('nav.login')  
    }}</div></router-link>  
</li>
```

ในธีมนัดนี้ใช้ข้อมูลชุดเดียวกัน: ลิงก์ `/pagemember` แสดง `memName` และปุ่มไอคอน `sky-logout` เรียก `memLogout`

9 รันและทดสอบทั้งระบบ

9.1 เปิดโปรแกรม (2 terminal)

Terminal 1 – Backend	bash
<pre>npm run server # API server is running at http://localhost:3000</pre>	
Terminal 2 – Frontend	bash
<pre>npm run dev # Local: http://localhost:5173/</pre>	

เปิดเว็บด้วย `http://localhost:5173` เท่านั้น

cookie ตั้งค่า `sameSite: strict` ถ้าเปิดด้วย `127.0.0.1:5173` browser จะมองว่า `localhost:3000` เป็นคนละเว็บ แล้วไม่ส่ง cookie ทำให้ล็อกอินแล้วหลุดทันที

9.2 รายการทดสอบ

ข้อความในตารางเป็นข้อความเมื่อเลือกภาษา ไทย (ปุ่ม EN / ไทย มุมบนขวา) ค่าเริ่มต้นของเว็บเป็นภาษาอังกฤษ

✓	ทำอะไร	ผลที่ควรได้
☐	เปิด <code>http://localhost:5173/</code> ตอนยังไม่ล็อกอิน	ถูกพาไป <code>/login</code>
☐	กด "ยังไม่มีบัญชี? ลงทะเบียนใหม่"	ไปหน้า <code>/register</code>
☐	กรอกอีเมล ชื่อ รหัสผ่าน แล้วกดตกลง	ข้อความ "ลงทะเบียนสำเร็จ" และมีแถวใหม่ในตาราง <code>members</code>
☐	สมัครด้วยอีเมลเดิมซ้ำ	ข้อความ "อีเมล ... ถูกใช้ลงทะเบียนแล้ว"
☐	ล็อกอินด้วยรหัสผิด	ข้อความ "อีเมลหรือรหัสผ่านไม่ถูกต้อง" และไม่มี cookie
☐	ล็อกอินด้วยรหัสถูก	ไปหน้า <code>/pagemember</code> เห็นอีเมล ชื่อ และ <code>member</code> เมนูแสดงชื่อสมาชิก
☐	F12 → Application → Cookies → <code>http://localhost:5173</code>	มี <code>token</code> ตี๊ก <code>HttpOnly</code> และ <code>Secure</code>
☐	F12 → Console พิมพ์ <code>document.cookie</code>	ไม่เห็น <code>token</code> (เพราะเป็น <code>httpOnly</code>)
☐	พิมพ์ URL <code>/login</code> ขณะล็อกอินอยู่	ถูกพากลับไป <code>/pagemember</code>
☐	รีเฟรชหน้า (F5)	ยังล็อกอินอยู่ (อ่านจาก cookie)
☐	กดออกจากระบบ แล้วกดยืนยัน	ไปหน้า <code>/login</code> , cookie <code>token</code> หายไป

☐	ลบ cookie ใน DevTools แล้วรีเฟรช	ถูกพาไป <code>/login</code>
--------------------------	----------------------------------	-----------------------------

10 ปัญหาที่พบบ่อยและวิธีแก้

อาการ / ข้อความ	สาเหตุ	วิธีแก้
หน้าเว็บขึ้น "เชื่อมต่อเซิร์ฟเวอร์ไม่สำเร็จ" / Server error	PostgreSQL ไม่ได้เปิด หรือค่าใน .env ผิด	เปิด service PostgreSQL ตรวจสอบ DB_* แล้วรีสตาร์ท <code>npm run server</code>
<code>relation "members" does not exist</code>	ยังไม่ได้สร้างตาราง หรือสร้างผิดฐานข้อมูล	ทำขั้นตอนที่ 3 ในฐานข้อมูลที่ตรงกับ DB_NAME
<code>column "memEmail" does not exist</code>	สร้างคอลัมน์โดยไม่มี " " จึงกลายเป็นตัวพิมพ์เล็ก	<code>DROP TABLE members;</code> แล้วสร้างใหม่ตามข้อ 3.4
<code>permission denied for table members</code>	user dev ไม่มีสิทธิ์	รัน <code>GRANT SELECT, INSERT, UPDATE, DELETE ON members TO dev;</code>
<code>password authentication failed for user</code>	DB_USER / DB_PASSWORD ไม่ตรง	แก้ .env ให้ตรงกับ PostgreSQL
server ไม่ขึ้น: SECRET_KEY is missing	ไม่มี SECRET_KEY ใน .env	เพิ่มค่าตามขั้นตอนที่ 4
ล็อกอินสำเร็จแต่แดงกลับหน้า login	browser ไม่เก็บ/ไม่ส่ง cookie	เปิดเว็บด้วย localhost (ไม่ใช่ 127.0.0.1) และตรวจว่ามี <code>withCredentials = true</code> กับ <code>credentials: true</code>
Console: blocked by CORS policy	origin ของเว็บไม่อยู่ใน whitelist	เพิ่ม origin นั้นใน <code>cors({ origin: [...] })</code> ของ server.js
<code>req.cookies</code> เป็น <code>undefined</code>	เรียก <code>cookieParser()</code> หลัง router	ย้าย <code>app.use(cookieParser())</code> ไปไว้ก่อน <code>app.use(memberRoutes)</code>
ใช้งานไปสักพักแล้วหลุด	token หมดอายุ 1 ชั่วโมง (<code>expiresIn: '1h'</code>)	ล็อกอินใหม่ ถ้าต้องการนานขึ้นให้แก้ทั้ง <code>expiresIn</code> และ <code>maxAge</code>
ปุ่ม "ข้ามการเข้าสู่ระบบ" ใช้แล้วรีเฟรชหลุด	เป็นการล็อกอินจำลองในหน่วยความจำ	ปกติ ปุ่มนี้มีเฉพาะตอน <code>npm run dev</code> และไม่มีใน build จริง

11 สิ่งที่ต่างจากเอกสารแลป

เอกสารแลป	โปรเจกต์นี้	เหตุผล
<code>bcrypt</code>	<code>bcryptjs</code>	มีในโปรเจกต์แล้ว API เหมือนกัน ไม่ต้องคอมไพล์บน Windows
ไฟล์ <code>index.js</code> , <code>routes/</code> , <code>services/database.js</code>	<code>server.js</code> , <code>src/router/</code> , <code>database.js</code>	โครงสร้างเดิมของโปรเจกต์
ตัวแปร <code>DBSERVER</code> , <code>DBUSER</code> , ...	<code>DB_HOST</code> , <code>DB_USER</code> , ...	ชื่อที่ <code>database.js</code> ใช้อยู่
<code>postMember</code> ส่ง <code>bodyData</code> กลับทั้งก้อน	ส่งกลับแค่อีเมล ชื่อ วันที่ และผล	ไม่ให้รหัสผ่านถูกส่งกลับไปใน response
ตรวจแค่ <code>memEmail</code> , <code>memName</code>	ตรวจ <code>password</code> ด้วย	ถ้าไม่มีรหัส <code>bcrypt.hash</code> จะ error
<code>SECRET_KEY='thailandgogo'</code>	ค่าสุ่ม 64 ตัวอักษร และ server ไม่ยอมเปิดถ้าไม่มี	เดา key ไม่ได้ = ปลอม token ไม่ได้
error ของ DB ส่ง <code>err.message</code> ตรงๆ	ถ้า message ว้างส่ง <code>Server error</code>	error ต่อ DB ไม่ได้มักไม่มีข้อความ ทำให้หน้าเว็บไม่แสดงอะไร
<code>withCredentials</code> ตั้งทุก component	ตั้งครั้งเดียวใน <code>main.js</code>	ไม่ต้องจำตั้งซ้ำ
แต่ละหน้าเรียก <code>/members/detail</code> เอง และ MainMenu ใช้ <code>watch</code>	<code>authStore.getMember()</code> ที่เดียว + router guard	ทุกหน้าเห็นข้อมูลชุดเดียวกัน ไม่ยิง API ซ้ำ
TheLogin เช็ก login ใน <code>onMounted</code>	router guard (<code>meta.guestOnly</code>)	ครอบคลุมทั้ง <code>/login</code> และ <code>/register</code>
หลัง logout ไป <code>/</code>	ไป <code>/login</code>	หน้าแรกของโปรเจกต์นี้ต้องล็อกอินก่อน
ฟอร์ม Bootstrap ตามแลป	ดีไซน์ใหม่ 2 ธีม + 2 ภาษา	ส่วนที่ออกแบบเพิ่มในโปรเจกต์

ระบบ `/auth/*` เดิม (ตาราง `users`) ยังอยู่ใน backend แต่หน้าเว็บไม่ได้เรียกใช้แล้ว และ API สิ้นค้า/ฐานข้อมูลยังไม่ได้ตรวจ token ซึ่งเอกสารแลปยังไม่ได้ครอบคลุมส่วนนี้